bubbled up first will never be updated again, and so heap's properties will be violated.

We could have two duplicates, call it `X` and `Y`, one being the children of the other: let's consider the case where `X` is a child of `Y`, and we call update to change their priority to a higher one. Inside `update`, after *updating* the two elements' priorities, there would have to be two calls to `bubbleUp`, once for `X`, and one for `Y`: the catch is that if we run them in the wrong way, we will end with an inconsistent heap, violating the heap's properties.

Suppose we bubble up `X`, the children, first: then it will immediately find its parent (which is `Y`) and stop (because it has the same value). When we call `bubbleUp(Y)` we will, though, discover that its parent now has lower priority, so we do need to move it towards the root. Unfortunately, we will never check `X` again, so it will never be moved up together with `Y` and heap's property won't be correctly reinstated.

Following a left-to-right order for the calls to `bubbleUp` will, instead, guarantees that the heap properties are properly reinstated.

As an alternative, we could change the conditions in `bubbleUp` and `pushDown`, and only stop when we find a strictly-higher-priority parent and strictly-lower-priority children respectively. Yet another alternative could be bubbling up (or pushing down) the nodes as we update them. Both these solutions, however, would be far from ideal for many reasons, last but not least performance-wise, as they would require a larger bound for the worst-case number